Software Architecture and Design Specification (SAD)

Remote Lab Reservation System (RLRS) — IEEE 1016 / 42010 Standard

Project	Remote Lab Reservation System (RLRS)
Version	1.1
Authors	Team 11: Mahipal Singh (PES1UG24AM153, Team Leader), Deepesh M (PES1UG24AM150), Lakshya Budhauliya (PES1UG24AM147), Venkat Kushal (PES1UG24AM124)
Date	11 November 2026
Status	Updated to match the implemented system (v1.1)

Revision History

Version	Date	Author	Change summary	Approval
1.0	30 October 2026	Team 11	Initial project-specific IEEE baseline specification.	Approved
1.1	11 November 2026	Team 11	Aligned with implemented v1.1 architecture: Layered modular monolith in Python stdlib + SQLite, embedded UML Component Diagram, UML Class Diagram, 2 UML Sequence Diagrams, and UML Activity Diagram.	Approved

Approvals

Role	Name	Signature / SRN / Email	Date
Project team representative	Mahipal Singh	PES1UG24AM153 (Team Leader)	11 November 2026
Scheduling & backend lead	Lakshya Budhauliya	PES1UG24AM147	11 November 2026
Calendar UI, auth & email lead	Deepesh M	PES1UG24AM150 (mdeepesh2006@gmail.com)	11 November 2026
QA & documentation member	Venkat Kushal	PES1UG24AM124	11 November 2026
Course instructor / TA	Kshitij Shetty	SE Teaching Assistant, Dept. of CSE(AIML)	Pending Review

Table of Contents

- 1. Introduction (1.1 Purpose, 1.2 Scope)
- 2. Architecture Goals and Constraints
- 3. Stakeholders and Concerns
- 4. Architecture and Components (4.1 Chosen Pattern, 4.2 Component Responsibilities, 4.3 UML Component Diagram)
- 5. Architecture Pattern and Technology Stack
- 6. Data Design & UML Class Model (6.1 Concurrency & Conflict Detection, 6.2 Reservation State Model, 6.3 Data Integrity & UTC Rules, 6.4 UML Class Diagram)
- 7. REST API / Service Design
- 8. Dynamic Design: UML Sequence & Activity Flows (8.1 Create Reservation Sequence, 8.2 Reminder Worker Sequence, 8.3 Maintenance Flow, 8.4 Booking Decision Activity Diagram)
- 9. Security Architecture
- 10. Error Handling, Logging and Monitoring
- 11. Architectural Traceability, Risks and Resolved Issues

1. Introduction

1.1 Purpose & 1.2 Scope

This Software Architecture and Design Specification (SAD) defines the architectural pattern, component structure, database schema, UML class model, REST API contracts, dynamic sequence/activity flows, and security controls of the Remote Lab Reservation System (RLRS) as implemented in v1.1.

2. Architecture Goals and Constraints

- Zero double booking under concurrency: conflict detection and insertion execute inside a single SQLite BEGIN IMMEDIATE write-locked transaction.
- Pure-logic testability: all scheduling, conflict, rule, reminder, maintenance, attendance, audit-redaction, and report calculations reside in rlrs_core/ with zero I/O dependencies.
- Exactly-once reminder durability: reminder and email jobs are persisted in SQLite with retry/backoff and idempotent state transitions.
- Cross-platform zero-install portability: built entirely on the Python 3.12+ standard library and vanilla HTML/CSS/JS.

3. Stakeholders and Concerns

Stakeholder	Primary Architectural Concern
Student	Responsive weekly availability calendar, immediate booking confirmation, reliable email reminders.
Lab In-charge	Lab-scoped control over restricted equipment approvals, operating hours, maintenance blocks, and attendance.
Administrator	Full-system visibility across labs, utilisation metrics, and tamper-evident audit logs.
Developers (Team 11) & TA	Clean separation between rlrs_core and rlrs_app, reproducible CI across Linux/Windows/macOS, full SRS traceability.

4. Architecture and Components

4.1 Chosen Pattern: Layered Modular Monolith

RLRS uses a layered modular-monolith architecture: a browser front end with a week-view calendar grid, a REST API dispatch layer, a transactional service layer delegating decisions to a pure domain/scheduling engine, and a SQLite persistence layer. A background worker thread started by the server process dispatches due reminders and queued emails every 30 seconds.

4.2 Component Responsibilities

Component	Implemented Module(s)	Responsibilities & Key Interactions
Frontend / Calendar UI	rlrs_app/static/index.html, app.js, style.css	Renders week-view availability grid, booking form, My Reservations, Manage and Reports tabs; calls REST API via fetch().
REST API Layer	rlrs_app/server.py	ThreadingHTTPServer + ROUTES regex table, JSON serialization, 1 MB body cap, static file serving with path-traversal guard, /api/health.
Auth & Security	rlrs_app/security.py, service.py	PBKDF2-HMAC-SHA256 (120k iterations), 8h HttpOnly session tokens, server-side RBAC, lab-ownership and holder-only remote_info checks.
Service & Persistence Layer	rlrs_app/service.py, db.py	Wraps pure rlrs_core logic in SQLite BEGIN IMMEDIATE transactions; converts lab-local times (+05:30 IST) to UTC for storage.
Scheduling Engine	rlrs_core/conflict.py, rlrs_core/rules.py	Pure half-open interval [start, end) conflict detection and 7-rule booking validation.
Operations & Reporting	rlrs_core/maintenance.py, attendance.py, report.py, audit.py	Pure domain logic for maintenance auto-cancellation, post-slot attendance, utilisation math, and secret-redacted auditing.
Notification & Worker	rlrs_core/reminders.py, rlrs_app/mailer.py, server.py	Background thread polling every 30s; sends reminders and queued emails via SmtpTransport or FileTransport (outbox/) with 3-attempt retry.

4.3 UML Component Diagram

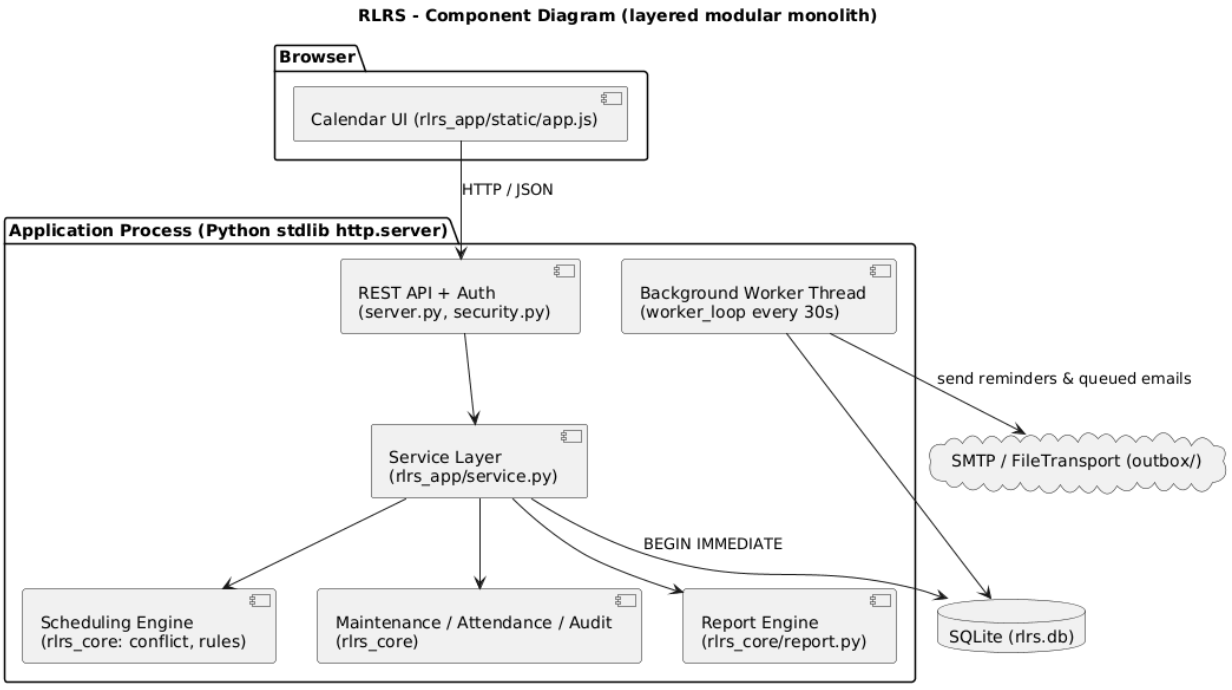


Figure 4.1: RLRS Layered Modular-Monolith Component Diagram (05_component.puml)

5. Architecture Pattern and Technology Stack

The implemented v1.1 stack uses the Python standard library (http.server, sqlite3, threading, smtplib, hashlib.pbkdf2_hmac) and a vanilla HTML/CSS/JS frontend, verified by pytest (71 unit/integration/API tests) and Playwright Chromium (9 UI steps) in GitHub Actions across Ubuntu, Windows, and macOS.

6. Data Design & UML Class Model

SQLite Table	Columns (as built in rlrs_app/db.py)	Integrity Rules
users	id, name, email (UNIQUE), role (Student|LabInCharge|Admin), pw_hash	Salted PBKDF2 hash; never store plaintext password.
sessions	token (PK), user_id (FK), expires (UTC)	8-hour TTL; delivered via HttpOnly, SameSite=Lax cookie.
labs	id, name, incharge_id (FK), open_time, close_time, slot_minutes, max_duration, advance_days, max_active	Stores per-lab operating hours and booking rule limits.
equipment	id, lab_id (FK), name, type, description, requires_approval, active, remote_info	Inactive equipment cannot be booked; remote_info restricted to Confirmed holders & staff.
reservations	id, user_id (FK), equipment_id (FK), start_ts (UTC), end_ts (UTC), status, created_at (UTC), reason	Status in (Pending, Confirmed, Cancelled, Rejected, Completed, NoShow).
maintenance	id, equipment_id (FK), start_ts (UTC), end_ts (UTC), reason	Creation cancels overlapping Pending/Confirmed reservations in the same transaction.
reminder_jobs	id, reservation_id (UNIQUE FK), send_at (UTC), status, attempts, next_attempt_at	UNIQUE(reservation_id) guarantees at-most-once reminder scheduling.
email_jobs	id, to_email, subject, body, status, attempts, next_attempt_at, created_at	Up to 3 delivery attempts with 5-minute backoff, then marked Failed.
audit	id, actor, action, at (UTC), details (JSON)	Append-only; password/token/secret keys automatically redacted.

6.1 Concurrency Lock, 6.2 State Model & 6.3 UTC Storage Rules

- Transactional Concurrency Guard (6.1): Every booking runs inside BEGIN IMMEDIATE, acquiring SQLite's write lock before reading existing reservations, validating rules/conflicts, and inserting the reservation, email_job, reminder_job, and audit row.
- Reservation Lifecycle State Model (6.2): (new) -> Confirmed (or Pending if requires_approval); Pending -> Confirmed | Rejected | Cancelled; Confirmed -> Cancelled (>60m before start for students, anytime for staff/maintenance) | Completed | NoShow (after slot end).
- UTC Persistence (6.3): All database timestamps are stored in UTC; service.py converts to/from lab-local time (+05:30 IST default via RLRS_UTC_OFFSET) at the persistence boundary.

6.4 UML Domain & Engine Class Diagram

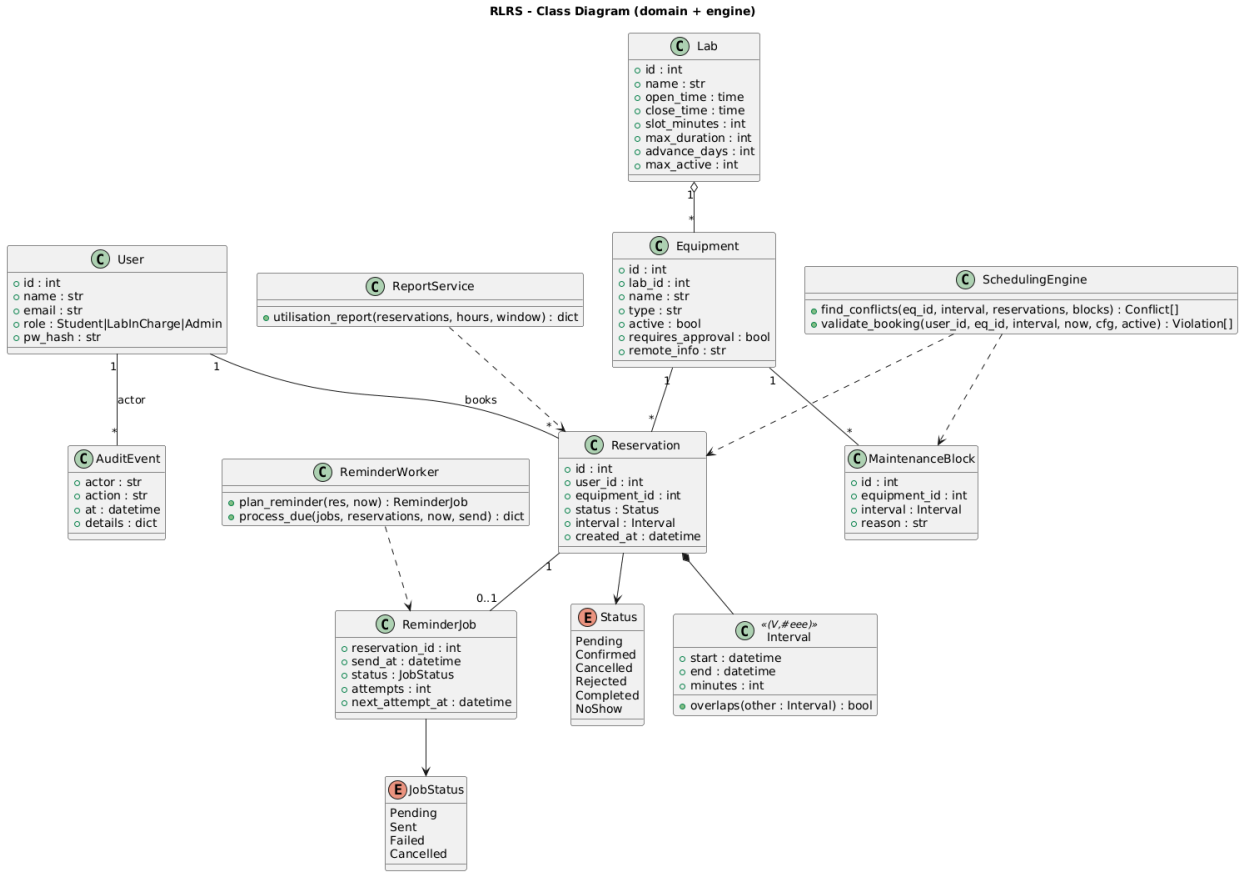


Figure 6.1: RLRS UML Class Diagram — Domain Entities and Pure Engine Services (02_class.puml)

7. REST API / Service Design

Operation	Method & Route	Auth	Response / Error Codes
Health check	GET /api/health	Public	200 {status: 'ok', db: 'ok', time}
Register / Login / Logout	POST /api/register, /api/login, /api/logout	Public	201 user / 200 + Set-Cookie / 200; 400 BAD_INPUT; 401 BAD_CREDENTIALS; 409 EMAIL_TAKEN
Current session user	GET /api/me	Signed-in	200 {id, name, email, role}; 401 UNAUTHENTICATED
List / Update labs	GET /api/labs; PUT /api/labs/{id}	Signed-in / Lab Staff	200 lab list or updated lab; 400 BAD_INPUT; 403 FORBIDDEN
List / Manage equipment	GET /api/equipment; POST /api/equipment; PUT /api/equipment/{id}	Signed-in / Lab Staff	200 list (remote_info filtered for students); 201 created; 403 FORBIDDEN
Equipment availability	GET /api/equipment/{id}/availability?from=&to;=	Signed-in	200 {equipment_id, slot_minutes, slots[]}; 400 BAD_RANGE / BAD_DATE
Create reservation	POST /api/reservations	Signed-in	201 Confirmed/Pending; 400 RULE_VIOLATION; 409 CONFLICT / EQUIPMENT_INACTIVE
My / Staff reservations	GET /api/reservations/mine; GET /api/reservations/manage	Signed-in / Lab Staff	200 reservation list (with remote_info on Confirmed)
Cancel reservation	POST /api/reservations/{id}/cancel	Owner or Lab Staff	200 Cancelled; 403 FORBIDDEN; 409 CUTOFF_PASSED / NOT_CANCELLABLE
Approve / Reject	POST /api/reservations/{id}/approve, /reject	Assigned Lab Staff	200 Confirmed/Rejected; 400 REASON_REQUIRED; 403 FORBIDDEN; 409 NOT_PENDING
Mark attendance	POST /api/reservations/{id}/attendance	Assigned Lab Staff	200 Completed/NoShow; 409 ATTENDANCE_REJECTED (before slot end)
Maintenance block	POST /api/maintenance	Assigned Lab Staff	200 {block_id, cancelled_reservation_ids[]}
Utilisation & Audit	GET /api/reports/utilisation?from=&to;=; GET /api/audit	Lab Staff	200 report metrics / latest 100 redacted audit entries; 403 FORBIDDEN

8. Dynamic Design: UML Sequence & Activity Flows

8.1 Create Reservation Sequence Diagram (RLRS-6)

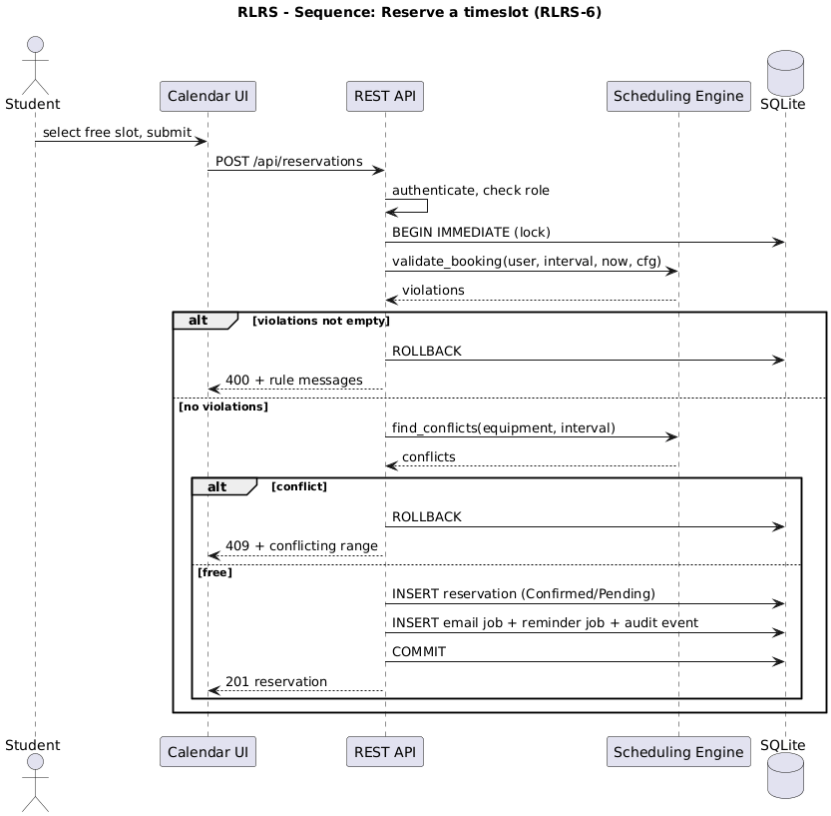


Figure 8.1: UML Sequence Diagram — Atomic Reservation Creation with Rule & Conflict Checks (03_sequence_booking.puml)

8.2 Automated Reminder Worker Sequence Diagram (RLRS-10)

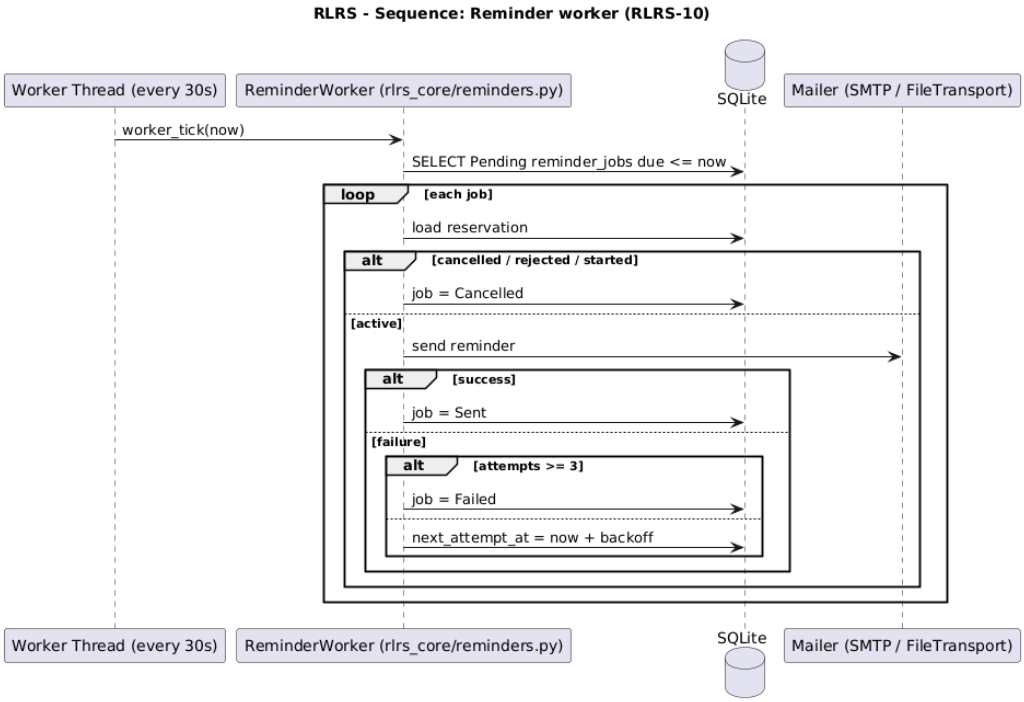


Figure 8.2: UML Sequence Diagram — Background Reminder Worker with Retry & Backoff (03_sequence_reminder.puml)

8.3 Booking Decision & Approval UML Activity Diagram

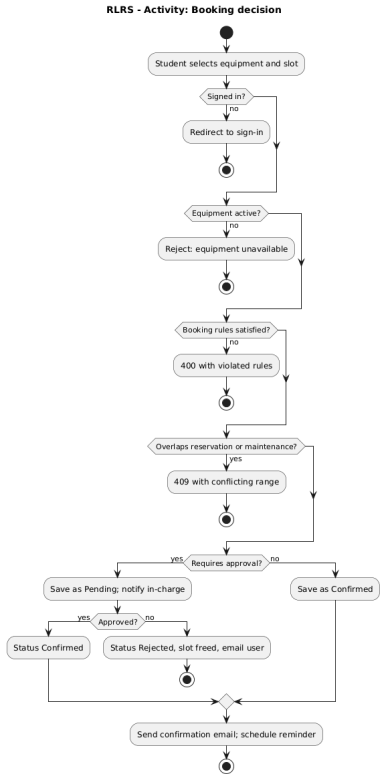


Figure 8.3: UML Activity Diagram — End-to-End Booking Decision & Approval Flow (04_activity_booking.puml)

9. Security Architecture

Threat / Concern (STRIDE)	Implemented Control in v1.1	Verification Status
Spoofing / Unauthorized access	PBKDF2-HMAC-SHA256 salted hashes (120k rounds); 8-hour random 256-bit token in HttpOnly, SameSite=Lax cookie.	Verified (TC-AUTH-01, TC-SEC-01)
Elevation of privilege	Server-side require_role and _require_lab_staff checks on every mutating route and staff query.	Verified (TC-AUTH-02, RLRS-11 cross-lab test)
Tampering / Acting on others' bookings	Owner-only cancellation check; lab in-charges restricted to their assigned lab_id.	Verified (TC-SEC-02)
Information disclosure (remote_info & errors)	remote_info returned to students only when holding a Confirmed reservation; 500 handler suppresses tracebacks.	Verified (TC-SEC-02, TC-ERR-01)
SQL Injection / XSS / Path traversal	100% parameterized SQLite queries; DOM built with textContent (zero innerHTML); static path resolution check.	Verified (TC-SEC-03, TC-SEC-04)
Repudiation	Append-only audit table with automatic secret redaction for login, booking, approval, maintenance, and attendance actions.	Verified (TC-AUDIT-01)

10. Error Handling, Logging and Monitoring

- All expected domain/validation errors raise ApiError(status, code, message, details) and serialize to structured JSON.
- Unexpected exceptions log to stderr and return HTTP 500 {'error': 'INTERNAL', 'message': 'Something went wrong.'} with zero stack-trace leakage.
- GET /api/health executes SELECT 1 on SQLite for automated liveness monitoring.

11. Architectural Traceability, Risks and Resolved Issues

SRS Requirement Group	Architectural & Design Realization
RLRS-F-001, F-002 / SR-001..003	rlrs_app/security.py + session cookie + role/lab guards in rlrs_app/service.py
RLRS-F-003..006	labs and equipment tables + service.availability + rlrs_app/static/app.js week grid
RLRS-F-007..009 / NF-002, NF-007	rlrs_core/conflict.py + rlrs_core/rules.py + BEGIN IMMEDIATE transaction in service.create_reservation
RLRS-F-010..013, F-018	Reservation state machine + rlrs_core/maintenance.py + rlrs_core/attendance.py
RLRS-F-014..016 / NF-003, NF-004	reminder_jobs & email_jobs tables + rlrs_core/reminders.py + worker_loop in server.py
RLRS-F-017 / SR-005	Confirmed-holder filtering in service.list_equipment and service._res_dict
RLRS-F-019, F-021 / SR-007	rlrs_core/report.py + rlrs_core/audit.py (with secret key redaction)